Intern Assignment: "Talk to Founder" — Voice AI Agent for Maneuver

Overview

Build a **Voice AI "Talk to Founder" web application** for Maneuver using the LiveKit Agents framework.

When a visitor lands on the app, they should be able to have a natural, real-time voice conversation with an AI agent that represents the founder. The agent has two jobs:

1. **Run a discovery call** — proactively ask the kinds of questions a founder would ask on a discovery call (who they are, what they're working on, problem they're trying to solve, timeline, budget, etc.) and capture that information.
2. **Answer questions about Maneuver** — if the visitor flips the script and asks about Maneuver (services, process, case studies, pricing model, team), the agent should answer accurately and conversationally.

Voice in → voice out. That's the baseline. The thing that will set a strong submission apart is the **synchronized visual layer** described below.

What we're evaluating

This assignment is intentionally open-ended. We're looking at:

- **Shipping ability** — does it actually work end-to-end?
- **Product instinct** — does the conversation *feel* like talking to a founder, not a chatbot?
- **Technical judgment** — clean architecture, sensible choices on STT/LLM/TTS, low latency
- **Polish** — UI, error states, what happens when the user is silent, interrupted, or rude
- **The visual layer (stretch)** — see "Bonus" below. This is where you can stand out.

A submission that nails the basics with a clean conversation is better than a half-broken submission with every bonus attempted. Prioritize accordingly.

Core requirements (must-have)

1. Voice conversation

- Real-time, low-latency voice in / voice out using LiveKit Agents (Python or Node.js — your call)
- STT → LLM → TTS pipeline (or a realtime/multimodal model — either is fine)
- Reasonable turn detection — the agent shouldn't talk over the user or freeze on natural pauses
- Handles interruptions gracefully

2. Two conversation modes (handled in a single agent)

- **Discovery mode (default):** the agent opens the conversation, introduces itself, and walks the user through discovery questions. Don't make it feel like a form — it should branch based on what the user says.
- **Q&A mode:** if the user asks a question about Maneuver, the agent answers from a knowledge base you create (a short markdown file with services, process, positioning, etc. — invent reasonable content; we'll evaluate the *system*, not your knowledge of Maneuver internals).
- The agent should fluidly switch between the two modes within a single call.

3. Capture discovery info

At the end of the call (or on demand), persist the captured info somewhere — JSON file, console output, a `/leads` endpoint, your call. Show us what got captured.

4. Frontend

A simple web app the user can open in their browser, grant mic permission, and start talking. The agent-starter-react template is a fine starting point.

5. README

- How to run it locally
- What models/providers you used and why
- What you'd do next if you had another week

Bonus: Synchronized visual layer (this is what we really want to see)

While the agent is talking, the **frontend should react to the conversation in real time**. Examples:

- User asks "what services do you offer?" → as the agent starts answering, the screen renders a slide/card list of Maneuver's services.

- User asks about a specific service → the screen zooms into that service with detail.
- User says "tell me about your process" → a process diagram appears.
- During discovery, captured fields (name, company, problem, timeline) populate live on the side panel as the user answers.

How to wire this up: the LiveKit Agents framework supports forwarding LLM tool calls to the frontend over RPC and state synchronization between agent and frontend. Define tools like `show_services_slide`, `show_service_detail(name)`, `update_lead_field(field, value)` etc. — when the LLM calls them, the frontend renders the corresponding UI.

A great submission will make the visuals appear *with* the voice, not after — feel free to optimistically render UI as soon as a tool is called, even before the agent finishes speaking.

Other things that would impress us (pick what excites you, don't do all)

- Persistent transcript view alongside the visuals
- Agent shows live "thinking / listening / speaking" state cleanly
- Multi-agent handoff (e.g. a "discovery" agent hands off to a "scheduling" agent when the user is ready to book a follow-up)
- A real follow-up email or Slack ping triggered at the end of the call with the captured info
- An admin/founder view that shows past calls and what was captured

---

Tech constraints

- **Framework:** LiveKit Agents (required). Beyond that, your choice.
- **Language:** Python or TypeScript for the agent. Frontend in whatever (React strongly suggested since the starter exists).
- **Models:** Use whatever STT/LLM/TTS combo you want. LiveKit Inference, OpenAI, Deepgram, Cartesia, ElevenLabs — all fine. Tell us in the README what you picked and why.
- **Hosting:** Local-only is completely fine. No need to deploy anywhere.

Deliverables

1. **GitHub repo** (public or invite us) with the full code

2. **README** as described above

3. **A 2–5 minute demo video** (Loom or any screen recording). **Hard cap: 5 minutes.** Anything longer won't be watched. In that time, cover:

- You having a real voice conversation with your agent (the core)
- The visual layer reacting in real time, if you built it
- A 30-second tour of how the pieces fit together

Local-only is completely fine — we just need to see it working on your machine.

4. **The captured discovery output** from your demo call (the JSON/file/whatever)

Timeline

You have **5 days** from receiving this assignment. We don't expect you to spend 40 hours on this — budget what feels right. A focused 10–15 hours of work is enough to do this well.

If you get stuck on something specific, email us. We'd rather you ask than spin.

How we'll review

We'll run your repo locally, have a real conversation with your agent, and then jump on a 30-minute call to discuss:

- Walk us through your architecture decisions
- What broke, what you'd change, what you'd build next
- A small live extension — we'll ask you to add one feature or fix one thing on the spot

---

Resources to get you started

- LiveKit Agents GitHub
- Voice AI quickstart
- agent-starter-python — Python agent template
- agent-starter-react — React frontend template
- Forwarding tool calls to the frontend (RPC) — key for the visual layer
- Agent state synchronization
- LiveKit Cloud has a free tier that's more than enough for this assignment

---

Have fun with this. The best submissions tend to come from people who treated it as a real product they wanted to use, not an assignment they wanted to finish.